

Panda: A Pretrained Forecast Model for Chaotic Dynamics

Paper Reading and Benchmark Investigation

Anuj Bhadbhade
Indian Institute of Science, Bengaluru

Supervisor: Prof. Flora Salim
University of New South Wales, Sydney

May 2026

Outline

- 1 Problem Statement
- 2 Background Theory
- 3 Dataset Construction
- 4 Model Architecture
- 5 Paper Results
- 6 Benchmark Investigation
- 7 Limitations and Future Directions

The Forecasting Problem for Chaotic Systems

Chaotic systems are everywhere:

- Fluid flows and turbulence
- Neural circuits and brain dynamics
- Electronic oscillator networks
- Mechanical linkages (double pendulum)
- Climate and weather systems

The fundamental difficulty:

Nearby trajectories diverge exponentially:

$$|\delta(t)| \approx |\delta(0)| e^{\lambda_1 t}, \quad \lambda_1 > 0$$

This is not a modelling failure, but a consequence of the mathematical structure.

Predictability Horizon

$$T_{\text{Lyap}} = \frac{1}{\lambda_1}$$

Beyond a few Lyapunov times, any finite-precision forecast becomes useless for *point prediction*.

Yet the *attractor geometry* and *invariant measure* remain well-defined.

What Existed Before Panda

Specialized Local Models

- Trained on a *single* system
- Excellent in-domain forecasting
- **Useless for any other system**
- Cannot generalize across attractors

General Time Series Foundation Models

- Trained on vast real-world corpora
- Retail, finance, weather data
- **No knowledge of dynamical structure**
- Treat chaos like stock prices

The Gap

Neither approach answers: **is there something universal about chaotic dynamics that a model could learn across many systems and transfer zero-shot to unseen ones?**

Panda's Core Contribution

Key Question

Can we train a **global forecast model** on diverse chaotic systems that generalizes **zero-shot** to systems never seen during training?

Four concrete contributions:

- 1 **Dataset:** 2×10^4 novel chaotic ODEs via evolutionary search
- 2 **Model:** Pretrained multivariate transformer achieving competitive zero-shot forecasts on real experimental data
- 3 **Architecture:** Three theory-motivated choices each contributing independently
- 4 **Emergence:** Zero-shot PDE forecasting despite training only on low-dimensional ODEs

Lai, Bao, Gilpin. ICLR 2026.

Attractors and Strange Attractors

Attractor: compact invariant set A that nearby trajectories converge to.

Strange attractor: attractor that is

- **Bounded:** trajectories stay finite
- **Aperiodic:** no trajectory repeats
- **Fractal:** non-integer dimension

Lorenz system (canonical example):

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z$$

At $\sigma = 10$, $\rho = 28$, $\beta = 8/3$: the butterfly attractor with $D_2 \approx 2.06$.

Key Invariants

- **Lyapunov spectrum** $\lambda_1 \geq \dots \geq \lambda_n$
- $\lambda_1 > 0$: chaos (divergence)
- $\lambda_2 = 0$: flow direction
- $\lambda_3 < 0$: contraction to attractor
- **Kaplan-Yorke dimension:**

$$D_{KY} = j + \frac{\sum_{i=1}^j \lambda_i}{|\lambda_{j+1}|}$$

- **Pesin's formula:**

$$h_{KS} = \sum_{\lambda_i > 0} \lambda_i$$

Takens' Embedding Theorem

In practice, we observe only a scalar measurement $y(x)$ of the full state $x \in \mathbb{R}^d$. Can we reconstruct the full attractor from this alone?

Theorem (Takens 1981)

The delay embedding map

$$\Phi_{y,\tau,k}(x) = (y(x), y(\phi^\tau(x)), \dots, y(\phi^{(k-1)\tau}(x)))$$

*is generically an **embedding** for $k \geq 2d + 1$.*

Consequence: the delay-embedded trajectory preserves all topological invariants: same Lyapunov exponents, same attractor geometry, same invariant measure.

Architectural Implication

A patch of P consecutive observations *is* a delay vector. Patching is not just an efficiency trick. It is a theoretically grounded representation of attractor state.

Koopman Operator and eDMD

Koopman operator $\mathcal{K}^t$ acts on observable functions:

$$(\mathcal{K}^t g)(x) = g(\phi^t(x))$$

- ϕ^t is nonlinear, but $\mathcal{K}^t$ is **linear**
- Nonlinearity absorbed into (infinite-dimensional) function space
- Koopman eigenfunctions evolve as pure exponentials

Extended DMD (eDMD): approximate $\mathcal{K}^t$ via a finite dictionary $\Psi(x) = [\psi_1(x), \dots, \psi_N(x)]^\top$:

$$\Psi(x_{n+1}) \approx \tilde{K} \Psi(x_n)$$

Standard eDMD Dictionaries

- **Polynomial features:**
Taylor-motivated, exact for polynomial flows
- **Random Fourier features:** kernel approximation (Rahimi & Recht 2007)

These are exactly what Panda uses in its dynamics embedding. The transformer then learns an approximate Koopman propagator in this lifted space.

Ergodic Theory: Why Diversity Matters

Birkhoff Ergodic Theorem: for an ergodic system, time averages equal space averages:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T g(\phi^t(x)) dt = \int g d\mu$$

A single long trajectory already samples the full invariant measure μ .

Pesin's formula: the entropy (information production rate) equals the sum of positive Lyapunov exponents.

Implication for training data:

- More trajectories from the *same* system $\rightarrow$ diminishing returns
- New *systems* provide qualitatively new topological information

Scaling Law Prediction

Performance should scale with the **number of unique systems**, not the number of trajectories per system.

This is confirmed empirically in the paper (Section 5.3) with exponent -0.42 — steeper than language model scaling laws.

Evolutionary Search for Chaotic Systems

Goal: 2×10^4 diverse, verified chaotic ODEs.

Four-step pipeline:

- 1 **Founding population:** 129 hand-curated ODEs (like Lorenz, Rössler, double pendulum)
- 2 **Mutation:** perturb parameters
 $\theta'_a \sim \mathcal{N}(\theta_a, \sigma)$
- 3 **Recombination:** skew-product coupling (driver $\rightarrow$ response)
- 4 **Selection:** battery of chaoticity tests

Why skew-product recombination?

The driver evolves independently. Chaoticity of the combined system is maintained. The response either synchronizes to the chaotic driver or adds its own chaos.

Selection Criteria

- No divergence to infinity
- No convergence to fixed point
- Chaos 0-1 test: $K \approx 1$
- No near-recurrences (limit cycles)
- Rosenstein: $\lambda_1 > 0$
- KPSS + ADF stationarity tests

Integration: Radau (5th order implicit RK),
 $\text{rtol} = 10^{-9}$, required for stiff systems.

Data Augmentations and Test Split

Three principled augmentations (applied to 20% of trajectories):

- 1 **Time-delay embedding:** $x_i(t) \rightarrow x_i(t - \tau_i)$
By Takens, diffeomorphic to original.
- 2 **Convex combinations:** $X \leftarrow MX$, rows of M from Dirichlet
Preserves attractor topology.
- 3 **Affine transforms:** $X \leftarrow AX + b$
Rescales and shifts. All dynamical invariants preserved.

These create equivalent dynamics in different coordinate systems.

Held-Out Test Set

- 20 of 129 founders withheld entirely
- Neither they nor their descendants appear in training
- Test set evolved from withheld founders: **9,300 systems**
- No shared genetic lineage with any training system

This is a strict train-test split preventing any data leakage through shared ancestry.

Design Philosophy: Weather vs Climate

Two forecasting objectives for chaos:

- **Predicting the climate:** forecast the invariant measure — the long-run statistical distribution over the attractor.
- **Predicting the weather:** short-term pointwise accuracy — correct at each specific timestep.

Standard causal decoders parrot: they repeat motifs from context. For ergodic systems with frequent near-recurrences this accidentally gets statistics right, but fails on pointwise accuracy.

Panda chooses the weather objective: encoder-only, fixed 128-step prediction horizon, maximising short-term pointwise accuracy.

Key Architectural Choice

Non-causal encoder: processes all 512 context steps simultaneously before predicting.

Attractor geometry is a *global* property of the trajectory. A causal decoder that only sees the past misses it.

Architecture: Patching and Dynamics Embedding

Patching (patch size $P = 16$, stride P):

$$T \in \mathbb{R}^{C \times 512} \rightarrow T_P \in \mathbb{R}^{C \times 32 \times 16}$$

- Practical: $256\times$ reduction in attention cost
- Theoretical: each patch *is* a Takens delay vector

Dynamics Embedding (eDMD dictionary):

$$E(P) = [P, \Phi(P), F(P)] \in \mathbb{R}^{C \times 512}$$

- $\Phi(P)$: random polynomial features (degree 2, 3)
- $F(P)$: random Fourier features
- **Weights fixed at init, never trained**

Why Fixed Random Features?

- Johnson-Lindenstrauss: random projections to high dimensions approximately preserve distances
- Rahimi & Recht: RFF approximate a specific kernel
- Training them risks overfitting the embedding to specific training attractors
- Analogy: reservoir computing — fixed random reservoir, only readout is trained

Architecture: Temporal and Channel Attention

Temporal Attention:

- Channel dim treated as batch dim
- Self-attention over 32 patches per channel
- Uses p-RoPE ($p = 0.75$): partial rotary positional encoding
- Time ordering carries dynamical meaning, so positional encoding is essential

Channel Attention:

- Transpose: attention over C channels per patch position
- **No positional encoding**: channels are an unordered set
- **Permutation equivariant** w.r.t. channels
- Generalizes from 3-channel training to any C at test time

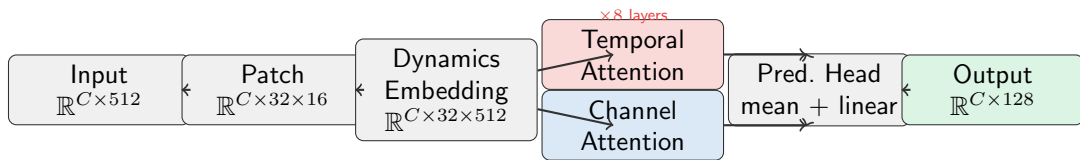
Why Channel Attention Matters

Chaotic systems have **strong channel coupling** — variables x, y, z are governed by coupled ODEs.

A univariate model forecasting each channel independently may match marginal statistics but **fails to respect joint attractor geometry**.

Channel attention captures inter-variable coupling explicitly.

Architecture: Full Forward Pass



Training details:

- 3 channels randomly sampled per trajectory (batching across dimensionalities)
- MSE loss, AdamW, cosine schedule
- 21M parameters total

Two output modes:

- **Forecast:** predict next 128 steps
- **MLM:** reconstruct 50% masked patches, learns global attractor structure

Zero-Shot Forecasting of Held-Out Systems

Setup:

- 9,300 held-out skew-product systems
- 6 context windows per system
- Prediction horizon: 128 steps
- Zero-shot: no fine-tuning at test time

Baselines (comparable scale):

- Chronos 20M (causal univariate)
- Chronos 20M SFT (fine-tuned on chaotic data)
- TimesFM 200M (10× larger, univariate)
- Time MOE 50M (univariate)
- DynaMix (recurrent, multivariate)

Key Results (Median sMAPE, $L = 128$)

Model	sMAPE
Panda 21M	27.6
Chronos 20M SFT	30.3
Chronos 200M	36.0
TimesFM 200M	> 40
DynaMix	47.1

Panda wins with **identical training data** as Chronos SFT. The advantage is architectural.

The Dynamical Systems Scaling Law

Experiment: fix total timepoints, vary diversity:

$$N_{\text{sys}} \times N_{\text{ics}} = \text{const}$$

Range: ($N_{\text{sys}} = 20000$, $N_{\text{ics}} = 1$) to ($N_{\text{sys}} = 156$, $N_{\text{ics}} = 128$).

Result: clean power law

$$\mathbb{E}[\text{sMAPE}] \approx 28.12 + 151.67 \cdot N_{\text{sys}}^{-0.42}, \quad R^2 = 0.99$$

More unique systems outperforms more trajectories per system.

Theoretical grounding: Birkhoff + Pesin.
Additional trajectories from the same system sample the same measure, new systems provide new topological information.

Distinct from Standard Scaling Laws

Standard neural scaling laws (Kaplan et al.) scale *total data volume* at fixed composition.

This law scales *diversity* at fixed data volume.

Exponent -0.42 vs -0.07 to -0.10 for LLMs. Steeper gains per new system, but faster saturation.

Performance not yet saturated at 2×10^4 systems.

Emergent PDE Forecasting

Setup: apply Panda zero-shot to two PDEs never seen during training:

- **Kuramoto-Sivashinsky:** 1D, 64 Fourier modes, $L = 100$
- **Von Kármán vortex street:** 2D vorticity, 512 PCA components

Baselines: FNO and DeepONet trained *in-distribution* on the specific PDE.

Result: Panda outperforms both baselines despite having never seen a PDE.

Why Does This Work?

- 1 **Inertial manifolds:** dissipative PDEs have finite-dimensional attractors (KS: $\dim \approx 10-20$) — comparable to training distribution
- 2 **Ruelle-Takens:** spatiotemporal chaos inherits geometric fingerprints of low-dim chaos
- 3 **Channel attention** captures spatial coupling between PCA modes the same way it captures ODE coupling

Long-Term Behavior and Failure Modes

Three failure modes at long horizons:

① **Panda: mean regression**

Predictions collapse toward zero beyond context window. KL divergence explodes to 15.25 at tail (vs 7.0 for Chronos SFT, 3.5 for DynaMix).

② **Chronos: parroting**

Repeats context motifs. Accidentally good on distributional metrics due to unstable periodic orbits.

③ **DynaMix: best long-term**

Recurrent hidden state integrates full history. No context window limit.

The Tradeoff

	Short horizon	Long horizon
Panda	Best	Poor
Chronos SFT	Good	Moderate
DynaMix	Poor	Best

Transformers win pointwise accuracy.
Recurrent models win distributional fidelity.
They are **complementary architectures**.

Experimental Design

Four experimental blocks:

- ① **Standard TS benchmarks** (ETTh1, ETTh2, Weather) — expected failure zone
- ② **Synthetic vs noisy**: double pendulum at graded noise levels
- ③ **Graded chaoticity**: Lorenz ρ sweep across bifurcation at $\rho = 24.74$
- ④ **Spatiotemporal**: 1D Burgers viscosity sweep; PCA vs spatial subsampling

Models: Panda 21M vs Chronos 20M (matched scale).

Primary metric: MAE (sMAPE unreliable on normalized data).

Context: 512 steps, prediction: 128 steps.

Result 1: Standard Time Series — Panda Does Not Win

ETTh1, ETTh2, Weather datasets:

- Strongly periodic with seasonal patterns
- No strange attractor
- Estimated $\lambda_1 \approx 0$ (non-chaotic)

Result: Both models perform similarly (MAE ≈ 0.7 - 0.9). Panda slightly worse.

Why:

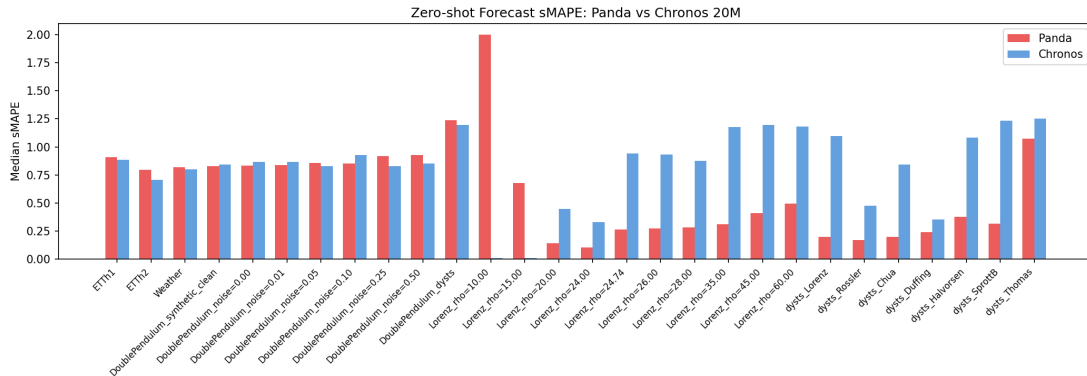
- Chronos trained on billions of real-world TS including exactly these patterns
- Panda trained on stationary chaotic ODEs — no seasonal inductive bias
- No attractor geometry to recover — channel attention provides no advantage

Finding 1

Panda is not a general-purpose forecasting model. On non-chaotic time series, it offers no advantage over standard foundation models.

This is expected and confirms domain-specificity.

Experiment Results: sMAPE Across All Datasets



Red = Panda Blue = Chronos

Panda wins on every chaotic system.
Both models comparable on ETT/Weather.

Result 2: Noise Robustness on Double Pendulum

Setup: simulate double pendulum at increasing Gaussian noise levels

$\sigma \in \{0, 0.01, 0.05, 0.10, 0.25, 0.50\}$ times signal std.

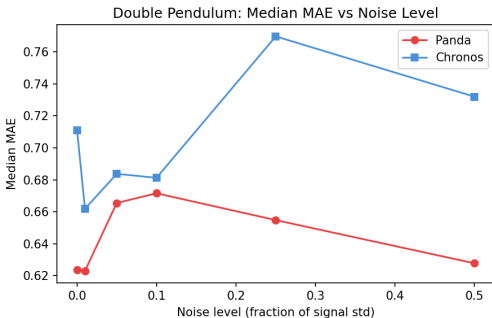
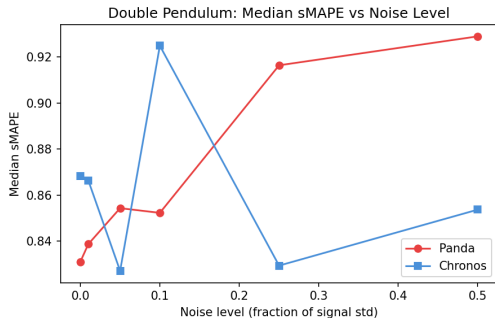
Result (MAE):

- Panda starts at 0.62 (clean) and stays ≈ 0.63 -0.67 across all noise levels
- Chronos starts at 0.71 and peaks at 0.77 at high noise
- **Panda consistently better at every noise level**
- **Panda's performance barely degrades with noise**

Why Noise-Robust?

- Polynomial + RFF features are smooth functions of input — small perturbations produce small embedding perturbations
- Takens' theorem is noise-tolerant: slightly delayed/noisy observations still reconstruct the attractor approximately
- Validates transfer to real experimental data (C. elegans, circuits, etc.)

Noise Robustness: Double Pendulum



Left: sMAPE (unreliable on normalized data — ignore oscillations). **Right:** MAE is the clean result. Panda MAE stays flat (≈ 0.62 – 0.67) across all noise levels. Chronos degrades more strongly and starts higher.

Result 3: The Chaos Threshold (Lorenz ρ Sweep)

Setup: vary Lorenz ρ from 10 (periodic) through 24.74 (onset of chaos) to 60.

Theoretical prediction:

At $\rho = 24.74$, first period-doubling cascade culminates in a strange attractor. λ_1 crosses zero here.

Results:

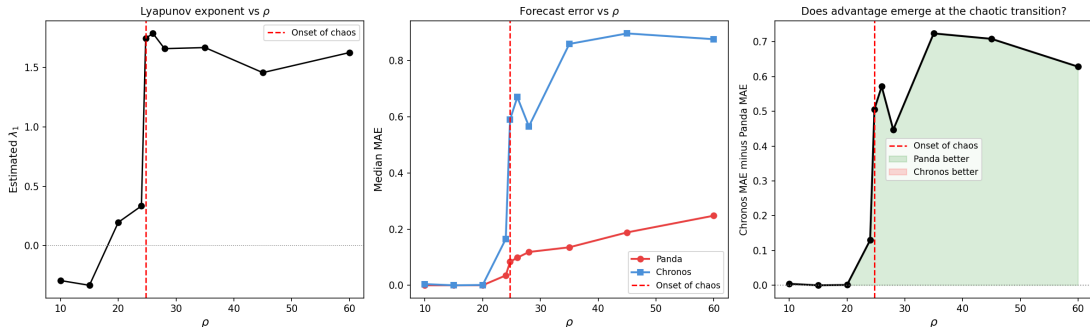
- $\rho < 24.74$: Panda advantage ≈ 0 throughout
- $\rho = 24.74$: advantage jumps sharply to ≈ 0.5
- $\rho > 24.74$: advantage stays positive across entire chaotic range (≈ 0.45 - 0.72)
- No chaotic ρ where Chronos wins

Finding 2

Panda's advantage is **exactly activated by the chaotic transition**. It is zero in the periodic regime and jumps at $\rho = 24.74$.

This is not a gradual trend, it is a **step function at the bifurcation point**, directly linking Panda's advantage to $\lambda_1 > 0$.

The Chaos Threshold: Lorenz ρ Sweep



Left: λ_1 crosses zero at $\rho = 24.74$ (onset of chaos, red dashed line). **Centre:** Chronos MAE jumps at the transition; Panda stays low. **Right:** Advantage is exactly zero before $\rho = 24.74$, then jumps and stays positive.

Result 4: Advantage vs Chaoticity Across All Systems

Setup: run Panda and Chronos on 30+ systems spanning dysts library, Lorenz rho sweep, double pendulum variants.

Result: Spearman $\rho = 0.37$, $p = 0.044$.
Positive correlation between λ_1 and Panda advantage.

Two sub-patterns:

- **Lorenz family (green):** above trend line — in-distribution, Panda has memorized the attractor geometry
- **Double pendulum (red):** below trend line — out-of-distribution topology (Lagrangian mechanics), but Panda still wins

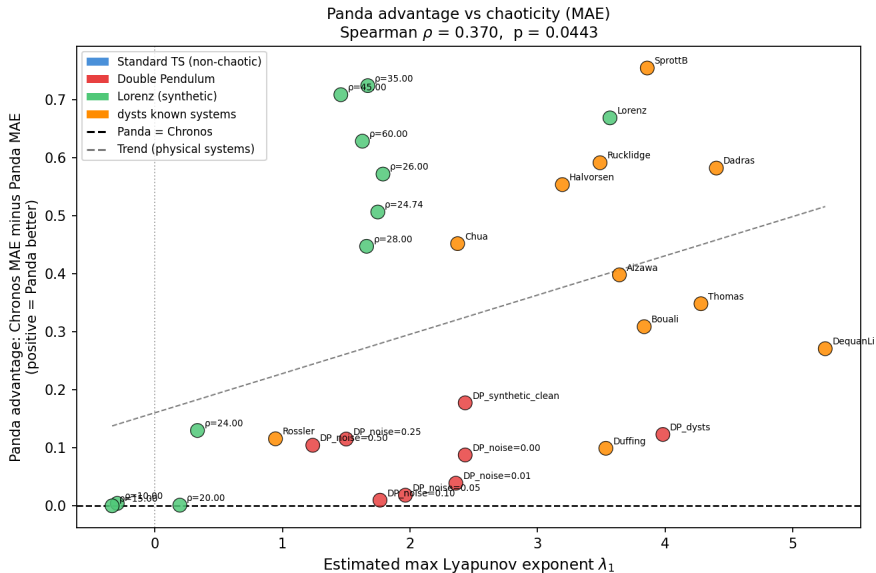
Finding 3

Chaos is **necessary and sufficient** for Panda's advantage.

Proximity to training distribution determines the *margin*, but does not determine *whether* Panda wins.

Panda has learned something **universal** about chaotic dynamics, not just specific attractors.

Advantage Correlates with Chaoticity (λ_1)



Result 5: Spatiotemporal — Burgers Viscosity Sweep

Setup: 1D Burgers equation $u_t + uu_x = \nu u_{xx}$ with varying viscosity. PCA to 16 channels.

- High ν : diffusion dominates, smooth non-chaotic solution
- Low ν : shock formation, spatiotemporal chaos

Results (Panda advantage in MAE):

ν	Panda	Chronos	Adv.
0.10	0.016	0.042	0.026
0.05	0.027	0.058	0.031
0.02	0.037	0.099	0.062
0.01	0.044	0.090	0.046
0.005	0.051	0.127	0.076

Finding 4

The chaos-threshold pattern **extends to PDEs**.

Panda advantage grows monotonically as viscosity decreases (chaos increases).

This is the spatiotemporal analogue of the Lorenz ρ sweep. Same phenomenon, higher-dimensional system.

Result 6: PCA vs Spatial Subsampling

Setup: same Burgers field ($\nu=0.005$), 16 channels, two representations:

- **Option A (PCA):** global spatial modes as channels
- **Option B (Subsampling):** actual spatial locations as channels

	Panda	Chronos	Adv.
PCA	0.122	0.215	0.093
Sub	0.015	0.059	0.044

Both models perform better on subsampling (easier absolute problem). **But Panda's relative advantage is larger with PCA.**

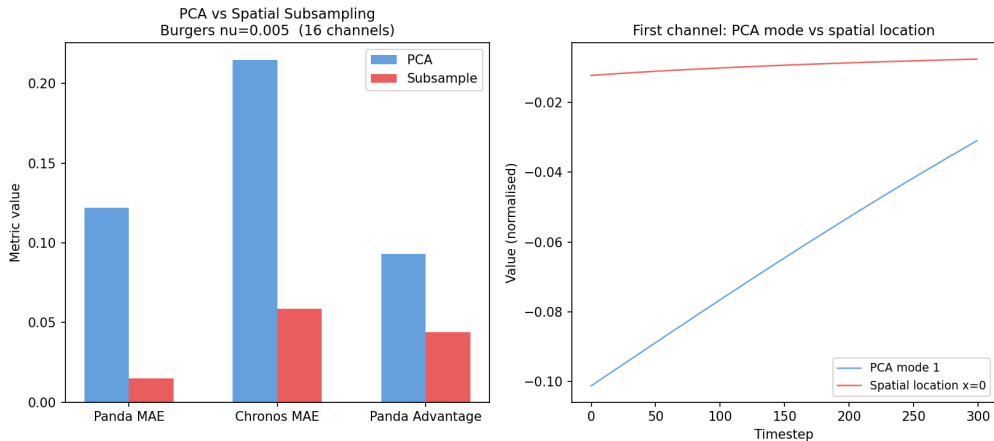
Interpretation

Subsampling artifact: spatial location $x = 0$ happened to be a near-nodal point; nearly constant field, trivial to forecast for both models.

PCA modes capture global energetic structure: harder to forecast, but the problem is more representative of true spatiotemporal dynamics.

Key insight: channel attention between PCA modes learns inter-modal Koopman coupling. Between spatial locations it must discover adjacency from scratch, spatial geometry is lost.

Spatiotemporal: PCA vs Spatial Subsampling



Left: PCA gives harder absolute problem (higher MAE) but **larger relative Panda advantage** (0.093 vs 0.044). **Right:** PCA mode 1 captures global energetic dynamics; spatial location $x=0$ is near-nodal, trivially predictable. Spectral/modal representations are more natural Koopman-space channels than raw spatial locations

What Panda Cannot Do

Current limitations:

- ① **Non-chaotic data:** no advantage on periodic, seasonal, or trending time series. Standard models (Chronos, TimesFM) dominate here.
- ② **Low-dimensional training:** trained on 3-10D ODEs. Inertial manifold argument breaks down for fully turbulent flows (λ_1 very large, high-dimensional attractor).
- ③ **Mean regression:** collapses to mean at very long horizons. Fundamental transformer context window limitation.
- ④ **No spatial awareness:** channel attention must rediscover spatial adjacency from data; spatial geometry is not explicitly encoded.

Panda vs Numerical Integrators

Integrators win: known equations + clean IC + short horizon. Radau at $\text{rtol}=10^{-9}$ tracks true trajectory to machine precision.

Panda wins:

- Unknown equations (only observations)
- Zero-shot generalization across systems
- Speed: 0.031s vs seconds for high-precision integration
- Noise tolerance up to 50% signal std
- Partial observability (Takens)

They solve **different problems** and are complementary.

How to Make Panda More Generalisable

Direction 1: Expand training distribution

- Include quasi-periodic systems (relax chaoticity filter slightly)
- Add spatiotemporal PDEs as training data (Burgers, KS, NS)
- Stochastic differential equations
- Higher-dimensional systems

Direction 2: Better pretraining objective

- MLM hurts rollout — what is the natural pretraining task for dynamics?
- Multi-task: MSE + spectral loss + invariant prediction
- Direct optimization of long-horizon distributional fidelity

Direction 3: Explicit spatial structure

- Channel attention discovers spatial coupling implicitly
- Better: encode spatial adjacency explicitly via custom attention masks
- Paper suggests this directly as future work

Direction 3

Can some inspiration be taken from DRIFT-Net?

The DRIFT-Net Connection

DRIFT-Net architecture:

- Dual branch: spectral (frequency domain) + image (spatial domain)
- Bandwise radial gating: learned spectral attention mask
- Designed for spatiotemporal reasoning tasks

What DRIFT-Net has that Panda lacks:

Explicit spatial structure: it knows channel 5 is spatially adjacent to channel 6.

What Panda has that DRIFT-Net lacks:

Temporal dynamics: Koopman embedding, cross-channel attention for dynamical coupling, chaotic attractor geometry.

Summary

What Panda is:

- A specialist global forecast model for chaotic dynamical systems
- Trained on 20,000 synthetic chaotic ODEs
- Zero-shot generalizes to unseen systems and PDEs
- 21M parameters, $60\times$ faster than Chronos

What we found:

- Panda advantage is **chaos-specific**
- Correlates with λ_1 (Spearman $\rho = 0.37$, $p = 0.044$)
- Extends to PDEs: Burgers viscosity sweep mirrors ρ sweep
- Noise-robust up to 50% signal noise

The Core Message

Chaos is necessary and sufficient for Panda's advantage.

Within the chaotic regime, proximity to training distribution determines the margin, but Panda wins on every chaotic system tested.

The path forward: combine Panda's temporal chaos model with DRIFT-Net's spatial structure for spatiotemporal reasoning.

Thank You